

Universitat Autònoma de Barcelona
Master Degree in Modelling for Science and Engineering

Implementation and Validation of an LLM-Based Planner for Modular ROS4HRI Robotic Systems

Master's Thesis

Student: Juan David Bendek Williamson
Programme: MSc Modelling for Science and Engineering
Institution / Lab: IIIA-CSIC / UAB
Supervisor: Raquel Ros
Academic year: 2025–2026
Repository: <https://github.com/ieverythng/nao-ros4hri-bridge>

June 17, 2026

Abstract

Large language models have shown strong potential for high-level reasoning, instruction understanding, and human-robot interaction. However, their integration into robotic systems remains challenging when language-driven decisions must be grounded in the robot’s actual state, validated before execution, and adapted when the environment or the user’s goals change. This thesis investigates the implementation and validation of a modular LLM-based planning architecture for ROS4HRI-compatible robotic systems, using the NAO robot as the main experimental platform.

The proposed system separates dialogue, symbolic grounding, task planning, and deterministic execution. A dialogue layer handles user-facing interaction and semantic intent declaration; a planner layer generates structured plans over an abstract skill registry; and an orchestrator validates and executes admissible actions while returning execution feedback to the planner. The work focuses on multi-intent instructions, planner supervision, failure handling, clarification requests, and the use of symbolic scene context through a knowledge and grounding pipeline.

The expected contribution is a reusable and inspectable planning framework for grounded human-robot interaction, together with an empirical evaluation of its robustness, modularity, and limitations. The thesis will compare direct execution against planner-mediated execution and analyse how abstract skill contracts, execution feedback, and grounded context influence task completion, replanning, and interaction quality.

Keywords: Human-Robot Interaction, ROS4HRI, Large Language Models, Task Planning, NAO Robot, Symbolic Grounding, Execution Monitoring, Replanning.

Acknowledgements

TODO:

Abbreviations

Term	Meaning
LLM	Large Language Model
HRI	Human-Robot Interaction
ROS	Robot Operating System
ROS4HRI	ROS-based Human-Robot Interaction framework
KB	Knowledge Base
WME	World Model Enricher
VLM	Vision-Language Model
VLA	Vision-Language-Action model
NAO	Humanoid robot used as the main experimental platform

Table 1: Initial list of abbreviations.

Contents

Abstract	i
Acknowledgements	ii
Abbreviations	iii
1 Introduction	1
1.1 Planning for HRI	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Research Questions	2
2 Background and Literature Review	3
2.1 Human-Robot Interaction and ROS	3
2.2 NAO Robot Stack	4
2.3 LLMs as High-Level Planners	4
2.4 Embodied Multimodal Models, VLMs, and VLAs	5
2.5 Symbolic State, Knowledge Bases, and Grounded Context	6
2.6 Execution Monitoring, Replanning, and Clarification	6
2.7 Literature Gap	7
3 System Requirements and Architectural	7
3.1 Functional Requirements	7
3.2 Design Principles	8
3.3 High-Level ROS Architecture	9
3.3.1 Runtime Flow	11
4 Runtime Contracts	12
4.1 Runtime Ownership Rules	12
4.2 Contract Map and Field Presentation	12
4.3 Contract 1: Chatbot Turn Output	13
5 Contract 2: Planner Request	15

5.1	Contract 3: Grounded Context	17
5.2	Grounded Context Pipeline	18
5.3	Planner LLM Internals	19
5.4	Contract 4: Planner Output	19
5.5	Contract 5: Execution Feedback	22
5.6	Contract 6: Planner Dialogue Act	22
6	Implementation	23
6.1	Repository Structure	23
6.2	Planner Engine	23
6.3	Goal-Keyed Supervisor	23
6.4	Skill Registry	24
6.5	Orchestrator and Execution Boundary	24
6.6	Grounding and Knowledge	24
6.7	Models and Runtime Configuration	24
7	Validation Methodology	24
7.1	Validation Goals	24
7.2	Scenario Matrix	25
7.3	Ablation Plan	26
7.4	Data Collection	26
8	Results	26
8.1	Plan Validity Results	26
8.2	Qualitative Scenario Traces	27
9	Discussion	27
9.1	Architectural Findings	27
9.2	Planner Behaviour	27
9.3	Grounding and World State	27
9.4	ROS4HRI Modularity	27
10	Limitations and Future Work	27

10.1 Current Limitations	27
10.2 Future Work	28
11 Conclusion	28
A Appendix A: Full Runtime Contracts	31
B Appendix B: Launch Commands	31
C Appendix C: Validation Log Template	31

1 Introduction

1.1 Planning for HRI

Successfully creating planning and execution systems for human-robot interaction (HRI) builds on a long tradition of automated planning. Classical planning languages such as the Planning Domain Definition Language (PDDL) were introduced to make planning domains, actions, preconditions, effects, initial states, and goals explicit and reusable across planners [1]. This formal view of planning shaped later hierarchical task network (HTN) approaches, where high-level tasks are decomposed into executable subtasks through domain-specific methods [2], [3], [4]. More recent neuro-symbolic and LLM-assisted planning work has revisited the same underlying problem from a different direction: how to connect flexible natural-language reasoning with grounded, verifiable action structures [5], [6], [7].

Large Language Models (LLMs) have therefore become promising natural-language operators for robot systems. They can transform user utterances into structured intent payloads, infer likely task goals, and propose multi-step action decompositions. In HRI, this is valuable because users rarely speak in planner-ready commands. They ask questions, refer to visible objects, change goals, and expect the robot to interpret context. At the same time, LLMs are not naturally grounded in the robot’s live scene state, skill availability, or execution constraints. A plan that is plausible in language may still refer to an unavailable object, use an unsupported action, or ignore a failed step.

This thesis focuses on the problem of integrating **LLM-based task planning** into a modular **ROS4HRI** robotic system centred on the NAO robot stack. The target is not to build an unconstrained prompt-to-action robot. The aim is to evaluate whether an LLM planner can operate more robustly when it is constrained by symbolic state, explicit skill contracts, deterministic validation, and execution feedback. In this architecture, natural language is used to interpret the user’s goal, but execution remains mediated by **ROS** interfaces, an orchestrator, and skill-level validation.

1.2 Problem Statement

Current **LLM-robot** integrations often face at least one of the following limitations:

- **they are tightly coupled to one robot or one task domain;**
- they rely on prompt-only action generation without strong execution validation;
- they do not clearly separate dialogue, planning, perception, and execution;
- they struggle with multi-intent instructions that unfold over time;
- they provide weak mechanisms for clarification, replanning, and failure recovery.

The central problem addressed in this thesis is therefore:

How can an LLM-based planner be integrated into a ROS4HRI robotic system so that it can generate and revise structured plans while preserving modularity, groundedness, and deterministic execution boundaries?

1.3 Objectives

The main objective of this investigation is to design, implement, and evaluate an LLM-based planner the NAO robot stack using principles from ROS4HRI. The planner is intended to extract user intent into known labels and natural language goals, generate structured skill plans, and remain grounded in the robot’s available capabilities, symbolic scene context, and execution feedback.

1. design and implement an LLM-based planning layer for a ROS-based robot stack;
2. define explicit runtime contracts between user intent extraction, planner admission, symbolic grounding, skill planning, execution, and feedback;
3. expose and discretize robot capabilities through an abstract skill registry instead of direct robot-specific APIs;
4. support multi-intent and multi-step user instructions through structured plan generation;
5. constrain planner outputs through deterministic validation before robot actions are dispatched;
6. implement execution feedback and planner supervision so the system can handle failed steps, ambiguous targets, clarification requests, replanning and changes in the world state;
7. evaluate the system through representative validation scenarios involving successful execution, failure, ambiguity, replanning, and user-facing responses.

1.4 Research Questions

This thesis is guided by the following research questions:

1. **RQ1:** Can a structured LLM planner enable a ROS4HRI robot to generate coherent multi-step skill plans from natural-language user requests?
2. **RQ2:** To what extent does compact symbolic grounding help the planner refer to objects, people, and scene context in a way that is consistent with the robot’s current world state?
3. **RQ3:** Does an abstract skill registry, combined with deterministic validation, reduce invalid planner outputs such as unsupported skills, malformed arguments, or robot-specific API calls?
4. **RQ4:** How does execution feedback affect the system’s ability to recover from failed actions, request clarification, or produce a revised plan when the world state

changes?

5. **RQ5:** Which architectural boundaries are required to keep LLM-based planning inspectable, modular, and safe enough for a ROS4HRI execution stack?

2 Background and Literature Review

2.1 Human-Robot Interaction and ROS

Human-robot interaction (HRI) studies how people and robots communicate, coordinate, and share space. In social and service robotics, the interaction is not only a command interface. The robot must interpret speech, gaze, gesture, people, objects, and task context while remaining physically safe and predictable. Early surveys in HRI and socially interactive robots describe this as a multidisciplinary problem that combines robotics, artificial intelligence, dialogue, perception, and interaction design [8], [9]. This motivates an architecture in which dialogue, perception, planning, and actuation are not treated as a single monolithic process.

ROS provides a practical software substrate for this separation. A ROS system is built from nodes that communicate through typed topics, services, and actions. Topics are suitable for streams such as perception or status updates, services for request-response queries, and actions for long-running behaviours that need feedback and cancellation. ROS 2 extends this model with design choices aimed at more robust distributed systems, including improved middleware support and deployment considerations for real robots [10], [11]. For this thesis, these boundaries are useful because they allow a language model to remain a high-level planner rather than a direct controller of robot hardware.

ROS4HRI builds on this idea by proposing reusable conventions and interfaces for HRI components in ROS. Mohamed and Lemaignan [12] identify the lack of a standard human representation as a barrier to reusable HRI pipelines, particularly when perception modules must represent bodies, faces, voices, groups, and social signals. Their ROS4HRI proposal is relevant here because it treats HRI perception as a set of interoperable streams rather than as robot-specific internal state. A planner should therefore receive compact task-relevant grounding, while perception and skill nodes retain access to richer evidence when they need to execute or validate an action.

This separation also matters for safety. A physical robot should not execute an action only because a language model produced plausible text. The runtime stack should validate that a requested skill exists, that its arguments are admissible, that the plan uses allowed interfaces, and that execution feedback is handled before further actions are dispatched. In this work, `nao_orchestrator` is kept as the deterministic execution boundary: it admits planner requests, validates plan steps, dispatches skills, and publishes execution feedback. The language model proposes structured plans, but it does not bypass the ROS execution layer.

2.2 NAO Robot Stack

The physical platform used in this thesis is NAO, a programmable humanoid robot originally developed by Aldebaran Robotics. Gouaillier et al. [13] describe NAO as an autonomous humanoid platform designed for research, education, and RoboCup-style experimentation. Its humanoid form, onboard perception pipeline, speech capabilities, and established use in academic robotics make it suitable for studying HRI behaviours such as spoken interaction, person/object grounding, gaze-oriented actions, navigation-like skills, and feedback-driven task execution. In this thesis, NAO is treated as the embodied execution platform, while the proposed architecture is designed so that the planning layer depends on abstract skills and ROS interfaces rather than on NAO-specific APIs.

2.3 LLMs as High-Level Planners

Large language models (LLMs) have been studied as high-level planners because they encode procedural and semantic knowledge that can be elicited from natural-language instructions. Huang et al. [14] show that language models can decompose high-level embodied tasks into action-like steps in simulated environments, but also show that raw language-model plans may not map cleanly to admissible actions. This problem is central for robotics: a plan that is linguistically reasonable can still be impossible, unsafe, or unsupported by the current robot.

Several approaches address this limitation by grounding the language model in a restricted action set. SayCan combines LLM task knowledge with affordance scores from pretrained skills so that proposed actions are both semantically appropriate and feasible for the robot [7]. Code as Policies uses language models to synthesize robot policy code over a provided API, which can express feedback loops and spatial computations, but still depends on a carefully defined set of callable primitives [15]. These approaches support the design choice used in this thesis: the LLM should plan over an explicit skill registry, not over arbitrary natural-language actions.

Reasoning and acting methods further show the value of coupling language output with external feedback. ReAct interleaves reasoning-oriented text with actions that query external sources or environments [16]. In robotics, Inner Monologue uses language feedback from perception, success detectors, and humans to improve long-horizon embodied task execution [17]. Reflexion extends the same general idea to language agents that improve across trials through verbalized feedback and memory, without updating model weights [18]. These works are relevant because they treat execution feedback as part of the planning process rather than as an afterthought.

This thesis adopts a narrower and more implementable form of closed-loop planning. The planner produces a structured skill plan, `nao_orchestrator` validates and executes the plan, skills return structured results, and `planner_llm` receives execution feedback. The planner may then request clarification, produce a revised plan, or explain failure. The

system does not expose private intermediate reasoning. It exposes only the final structured plan, dialogue acts, and feedback-dependent decisions. This preserves inspectability while avoiding the risks of treating free-form reasoning text as an execution contract.

Prompt-only planning remains limited. A prompt can tell an LLM which skills exist, but it cannot guarantee that the model will always select valid skills, use valid arguments, or remain grounded in the current scene. The stack therefore uses deterministic validation around the LLM. The planner output is JSON, plan steps are checked against allowed skills, recovery policies are constrained, and execution remains bound to the orchestrator. This is the main architectural distinction between using an LLM as a planner and using an LLM as a controller.

2.4 Embodied Multimodal Models, VLMs, and VLAs

Vision-language models (VLMs) and vision-language-action models (VLAs) provide an important context for this work. VLMs connect visual observations with language, while VLAs go further by mapping visual and language inputs to robot actions. BLIP-2 shows how frozen vision encoders and frozen language models can be connected through a lightweight transformer to support vision-language generation [19]. LLaVA uses visual instruction tuning to create a multimodal assistant capable of following image-based instructions [20]. These systems are not robot controllers, but they show how visual context can be converted into language-compatible representations.

Embodied multimodal models extend this direction into robotics. PaLM-E incorporates visual and continuous sensor inputs into a language model and evaluates the resulting model on embodied tasks including robotic planning and visual question answering [21]. RT-2 formulates robotic actions as tokens and co-fine-tunes vision-language models on web-scale data and robot trajectories, giving the model a direct path from visual observations and instructions to action outputs [22]. OpenVLA makes this family of models more accessible by releasing an open-source VLA trained on a large collection of robot demonstrations [23].

The present thesis does not attempt to train an end-to-end VLA policy. That would require a different experimental design, robot demonstration data, and low-level control evaluation. The contribution here is instead a modular high-level planning architecture for a ROS4HRI-based NAO stack. VLM and VLA literature is used to motivate future extensions in which richer visual or embodied state could improve the planner’s world model. In the current system, visual and KB evidence are converted into compact symbolic context before reaching the planner. This keeps the planner portable across robots, provided that the robot exposes equivalent skills and grounding interfaces.

2.5 Symbolic State, Knowledge Bases, and Grounded Context

Robotic planning requires some representation of the world. Pure perception streams are usually too detailed and unstable for high-level planning, while pure language descriptions risk being incomplete or stale. Symbolic knowledge bases provide one way to bridge this gap. KnowRob represents objects, actions, environments, and task knowledge in a form that supports robot reasoning and querying [24], [25]. RoboBrain similarly treats robotic knowledge as a multi-source resource that can support grounding, perception, and planning [26]. Other work has explored how interaction with users can help robots acquire knowledge about their environment and ground referential expressions [27], [28].

The current implementation follows the same general principle, but at a smaller scale. Detector-derived observations and KB rows are projected into a compact `grounded_context`. This context contains stable entity handles, entity kind, semantic class, visibility, and a bounded set of relevant relations. The planner can then refer to entities such as `cup_1` or `anonymous_person_ehfbf` without needing raw detector scores, image coordinates, or full RDF-style triples in every prompt.

This distinction is important for HRI. A user may ask “look at the person”, “find the cup”, or “what do you see?”. The robot must connect those phrases to current entities, but the planner does not need to subscribe directly to every perception stream. Skills can still requery raw scene or KB evidence at execution time. This allows the planner prompt to remain compact while preserving access to richer evidence where it is needed. A future World Model Enricher can extend this by maintaining a more explicit evolving scene state, including uncertainty, source recency, and relations between people, objects, and actions. The current system prepares for that extension by keeping grounding as a separate contract rather than embedding it inside the planner logic.

2.6 Execution Monitoring, Replanning, and Clarification

Planning in a physical robot system cannot stop at plan generation. Actions may fail, targets may disappear, the user may change the goal, and the robot may need clarification before continuing. Closed-loop methods in language-agent and embodied-planning literature support this view. Inner Monologue shows that feedback from the environment and humans can improve embodied task completion [17]. ReAct shows that language models can combine action with information gathering from external tools or environments [16]. Reflexion shows that feedback can be stored and reused as language-level experience across trials [18].

The implementation in this thesis uses execution feedback as a runtime contract. A skill result is not only a log message. It informs `planner_llm` whether a plan step succeeded, failed, needs user input, or should trigger replanning. The orchestrator never creates a replacement plan locally. If a step fails with a recovery policy such as `replan`, `nao_orchestrator` blocks unsafe continuation, publishes feedback, and waits for a revised

validated plan before dispatching further actions. This maintains a clear separation between execution authority and planning supervision.

Clarification is treated as part of the same loop. If the planner cannot safely select a target, or if execution feedback reports ambiguity, the planner emits a dialogue act rather than inventing a missing fact. The dialogue manager then realizes that act as speech. This makes user interaction part of the execution loop, not a separate fallback. The same mechanism can be used when the user’s goal changes during execution, or when the current world state no longer satisfies the preconditions for the next planned action.

2.7 Literature Gap

Existing work demonstrates several important components of language-guided robotics. LLMs can decompose tasks into action-like sequences [14]. Affordance-grounded methods can restrict language-model proposals to feasible robot skills [7]. Multimodal and VLA systems can connect vision, language, and action at larger scale [21], [22], [23]. Knowledge-base systems show how symbolic state can support grounding and planning [24], [26]. Execution-feedback approaches show that planning improves when the model can react to environment or human feedback [16], [17].

The gap addressed in this thesis is practical and architectural. We are aware of no existing equivalent ROS4HRI-oriented implementation that combines dialogue routing, compact symbolic grounding, LLM-based high-level planning, deterministic orchestrator validation, skill-level execution, and feedback-driven replanning in a single modular NAO stack. The aim is not to replace ROS execution with a language model. It is to place the language model at the planning and supervision layer while preserving ROS boundaries for perception, skills, validation, and speech realization.

The resulting architecture is positioned between two extremes. It is more flexible than a fixed intent-to-skill mapping because it can produce multi-step plans and react to feedback. It is more controlled than an end-to-end language or VLA policy because it exposes explicit runtime contracts and keeps execution authority inside `nao_orchestrator`. This makes it suitable for a master’s thesis implementation, while leaving a clear path toward richer world modelling and multimodal grounding in future work.

3 System Requirements and Architectural

3.1 Functional Requirements

This section defines the requirements and design principles that guide the proposed planner stack. The aim is to establish the architectural boundaries that the implementation must satisfy: natural-language interaction should remain separate from executable robot

control, LLM-generated plans should be validated before dispatch, and live execution feedback should be returned to the planner when the world state or task status changes. Section 5 has detailed node implementation

ID	Requirement
FR1	The system shall accept natural-language user input through the dialogue stack.
FR2	The system shall distinguish dialogue-only turns from execution-eligible turns.
FR3	The system shall convert execution-eligible user turns into admitted planner requests before invoking the planner.
FR4	The planner shall generate structured skill plans over an explicit skill registry.
FR5	The orchestrator shall validate planner outputs before any robot skill is dispatched.
FR6	The system shall publish execution feedback after relevant plan and step/skill events.
FR7	The planner shall support asking for clarification, replanning and plan failure when execution feedback indicates that the current plan cannot continue safely.
FR8	The system shall expose traces, logs, or runtime messages suitable for validation of the stack.

Table 2: Functional requirements for the proposed ROS4HRI planner stack.

3.2 Design Principles

1. **Dialogue must remain distinct from execution.** Dialogue-based turns such as greetings, conversation and queries about the robot’s world state should be remain as dialogue and must not be routed to execution unless the user explicitly requests an action.
2. **Planner doesn’t execute, it proposes structured skill plans:** `planner_llm` selects skills from the allowed registry and produces a structured plan based on the grounded scene context. It does not execute skills, speak directly, or call robot-specific APIs.
3. **The orchestrator remains the execution gate:** all robot execution is validated and dispatched downstream by `nao_orchestrator`. It admits planner requests, validates plans, dispatches skills, and publishes execution feedback.
4. **Skills own live evidence.** Perception and execution skills should query the Knowledge Base (KB), or robot state when needed. Final execution evidence remains skill-local.

3.3 High-Level ROS Architecture

The proposed architecture separates the HRI stack into interpretation, planning, grounding, execution, and feedback responsibilities, enabling a clear distinction between conversational reasoning, task-level decision making, deterministic execution, and environment-aware adaptation.

- **dialogue_manager** acts as the central dialogue ownership component, maintaining conversational state, coordinating turn-taking, managing interaction flow, and serving as the integration point between user-facing dialogue and downstream planning services.
- **chatbot_llm** is responsible for robot utterance generation, intent declaration and goal extraction during planner-relevant turns. It transforms natural language interactions into structured representations that depict the nature of the interaction. **Dialogue** and **knowledge query** turns remain conversational, **execution** turns trigger a planner request.
- **planner_llm** performs proactive task planning over the available skill registry, normalized user intents, dialogue context, and current robot world state. The planner reasons over high-level objectives, decomposes user requests into executable actions, and generates candidate plans that align with both robot capabilities and current world state constraints.
- **nao_orchestrator** provides deterministic plan validation and skill execution management. It verifies planner outputs against available skills and execution constraints, coordinates skill invocation, monitors execution progress, and emits structured execution feedback that can be consumed by both the dialogue and planning layers.
- **grounded_context** represents a detector-to-KB JSON object by linking perception outputs to semantically meaningful entities within the robot knowledge base. This grounding layer provides a shared representation of objects, locations, and contextual information that is consumed across the chatbot, planner and execution components.
- **skill_servers** provide the execution interface that call lower-level ROS components and robot capabilities. These servers encapsulate perception, navigation, manipulation, and interaction robot actions. Skill servers allow the planner to invoke complex robot actions with structured plans. They consume raw knowledge-base updates originating from perception pipelines and execution feedback, ensuring that newly observed entities, spatial data, state changes, and environmental information for subsequent task execution.

The interaction between these packages establishes a layered architecture in which conversational understanding produces structured intents, planning generates task-level strategies, grounding enriches decisions with environmental knowledge, orchestration ensures

safe and deterministic execution, and feedback mechanisms continuously update the system state for subsequent reasoning cycles.

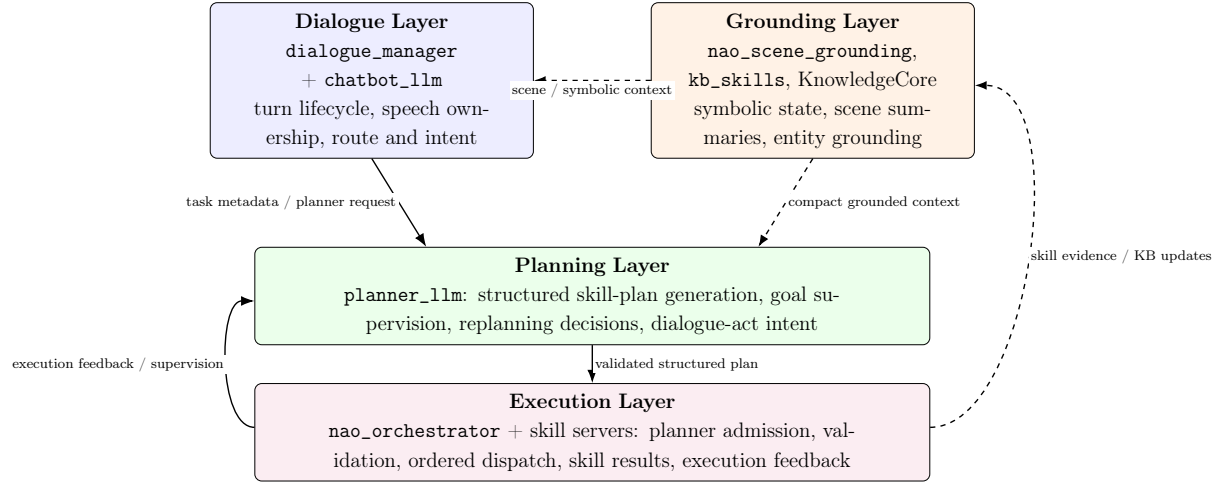


Figure 1: High-level layered architecture. Dialogue interpretation and symbolic grounding provide parallel inputs to the planning layer. The chatbot layer also consumes symbolic scene context. The planner proposes structured skill plans, while the execution layer validates and dispatches skills before returning feedback for supervision.

3.3.1 Runtime Flow

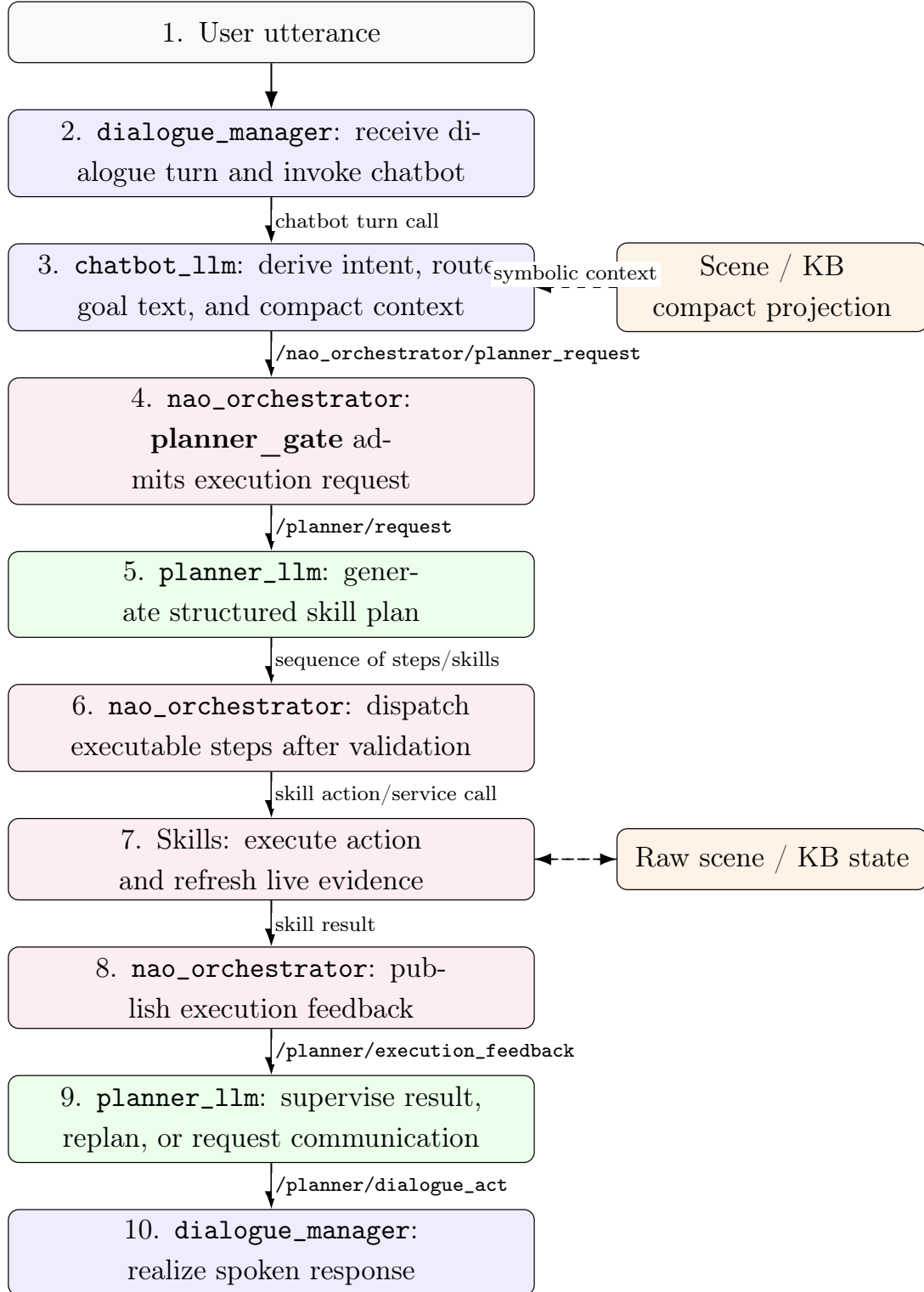


Figure 2: Runtime message and responsibility sequence from user input to spoken response. The colour coding follows Figure 1: dialogue components are blue, grounding and evidence are orange, planning is green, and execution is purple.

4 Runtime Contracts

The architecture relies on explicit runtime contracts between dialogue interpretation, planner admission, grounding, plan generation, execution, feedback, and speech realization. These contracts are used to keep the LLM-facing parts of the system inspectable while preserving deterministic control over robot execution. Each contract defines what is exchanged, which node produces it, which node consumes it, and where validation or ownership should occur. This is important because the planner does not control the robot directly: `chatbot_llm` interprets the user turn, `nao_orchestrator` admits and validates execution requests, `planner_llm` proposes structured skill plans, skills execute with live evidence, and `dialogue_manager` realizes user-facing speech. The table below gives the contract map used throughout the rest of this section. Later subsections describe the main payloads in more detail.

4.1 Runtime Ownership Rules

- `dialogue_manager` owns the dialogue turn lifecycle and user-facing speech realization.
- `chatbot_llm` owns user-facing semantic interpretation: intent derivation, route selection (dialogue or execution) and goal text for planner.
- `nao_orchestrator` owns deterministic planner admission, request normalization, executable-step validation, dispatch, step status, and feedback publication.
- `planner_llm` owns structured skill-plan generation, plan supervision, replanning decisions, and dialogue-act intent.
- Skills own live interaction with the robot, scene, and KB evidence required to complete their operation truthfully.

4.2 Contract Map and Field Presentation

Table 3 lists the active runtime contracts.

Table 3: Runtime contract map and ownership in the active planner stack.

ID	Contract	Interface / payload	Ownership
1	Chatbot turn output	Internal chatbot result before ROS planner publication.	<code>dialogue_manager</code> invokes the chatbot path; <code>chatbot_llm</code> produces the semantic result.
2	Planner request and admission	Candidate topic <code>/nao_orchestrator/planner_request</code> . Admitted topic <code>/planner/request</code> . Planner-facing payload is carried in <code>Intent.data</code> .	<code>chatbot_llm</code> proposes the execution request; <code>nao_orchestrator</code> admits and normalizes it; <code>planner_llm</code> consumes the admitted request.
3	Compact grounding	<code>grounded_context</code> inside Contract 2.	Scene and KB projection provide compact symbolic context for <code>chatbot_llm</code> and <code>planner_llm</code> .
4	Planner output	Topic <code>/intents</code> . Structured plan payload in <code>Intent.data.plan</code> .	<code>planner_llm</code> produces the plan; <code>nao_orchestrator</code> validates and dispatches executable steps.
5	Skill execution	ROS skill actions or services.	<code>nao_orchestrator</code> dispatches skill calls; skill servers execute robot or simulated actions.
6	Execution feedback	Topic <code>/planner/execution_feedback</code> . Structured JSON feedback payload.	<code>nao_orchestrator</code> publishes feedback; <code>planner_llm</code> uses it for supervision, replanning, clarification, or failure handling.
7	Planner dialogue act	Topic <code>/planner/dialogue_act</code> . Structured communication request.	<code>planner_llm</code> emits dialogue-act intent; <code>dialogue_manager</code> realizes the final speech.
8	Raw evidence	Topics and sources include <code>/scene/summary</code> , <code>/kb/query</code> , knowledge snapshots, skill results, and action evidence.	Scene, KB, and skill nodes provide evidence for compact projection, skill-local checks, and later feedback.

The table above lists only the contract and its meaning. Compact planner-facing fields are kept in the tables, while additional runtime metadata is described in prose when it is relevant for orchestration, planner supervision, tracing, or debugging.

4.3 Contract 1: Chatbot Turn Output

The intent in this contract is the user’s goal as extracted by the chatbot. It is not a planner output.

```

1 {
2   "verbal_ack": "Okay, I will do that.",
3   "route": "execution",
4   "confidence": 0.82,
5   "user_intent": {
6     "type": "look_at",
7     "goal_text": "look at the person",
8     "scene_targets": ["person"],
9     "request_kind": "new_goal"
10  }
11 }
```

Table 4: Fields in Contract 1: chatbot turn output.

Field	Meaning
<code>verbal_ack</code>	Optional immediate acknowledgement generated by the chatbot layer. This may be spoken or logged before execution begins, depending on the dialogue policy.
<code>route</code>	Turn classification produced by <code>chatbot_llm</code> . Supported values include <code>dialogue</code> , <code>knowledge_query</code> , and <code>execution</code> . Only <code>execution</code> turns enter the planner path; <code>dialogue</code> and <code>knowledge_query</code> remain non-execution turns but are still useful for tracing.
<code>confidence</code>	Confidence assigned by the chatbot or intent-extraction stage. It is useful for debugging and possible admission policy, but it is not a planner priority score.
<code>user_intent.type</code>	Normalized user-intent label extracted by <code>chatbot_llm</code> , such as <code>look_at</code> , <code>find_object</code> , <code>scan</code> , or <code>navigate_to</code> .
<code>user_intent.goal_text</code>	Natural-language objective passed toward the planner path as the task to satisfy. This field preserves the user’s goal in a compact form.
<code>user_intent.scene_targets</code>	Optional target hints extracted from the utterance, such as <code>person</code> , <code>cup</code> , or <code>table</code> . These are hints, not validated grounded entities.
<code>user_intent.request_kind</code>	Proposed goal transition class, such as <code>new_goal</code> , <code>update_goal</code> , <code>answer_to_clarification</code> , or <code>cancel_goal</code> . The planner gate inside <code>nao_orchestrator</code> should normalize and admit this field before publishing <code>/planner/request</code> .

5 Contract 2: Planner Request

The planner request is the task description generated by the orchestrator from the chatbot output and consumed by the planner

18

```

1 {
2   "goal_id": "goal_turn_123",
3   "request_kind": "new_goal",
4   "goal_text": "look at the person and report completion",
5   "normalized_intents": ["look_at"],
6   "grounded_context": {
7     "entities": [
8       {
9         "id": "anonymous_person_ehfbf",
10        "label": null,
11        "kind": "person",
12        "class": "Human",
13        "visible": true,
14        "relations": []
15      }
16    ]
17  }
18 }
```

““latex

Table 5: Fields in Contract 2: minimal planner-facing request.

Field	Meaning
<code>goal_id</code>	Identifier for the active objective. It groups replans and feedback for the same task.
<code>request_kind</code>	States whether this is a new goal, an update to the current goal, a cancellation, or another supported transition. <code>chatbot_llm</code> may propose it, but <code>nao_orchestrator</code> normalizes and admits it before publishing <code>/planner/request</code> .
<code>goal_text</code>	Objective extracted from the user utterance. This is the central planning input for <code>planner_llm</code> .
<code>normalized_intents</code>	Intent labels extracted by <code>chatbot_llm</code> . They help <code>planner_llm</code> choose suitable skills from the available skill registry.
<code>grounded_context</code>	Compact scene graph used by the planner as the current symbolic state of the world. It contains validated entities and relations rather than raw detector output.

““

Implementation metadata can still exist on the ROS envelope or runtime trace. Examples include request identifiers, dialogue turn identifiers, planner mode, parent goal ID, or superseded goal ID. These fields are useful for tracing, debugging, and goal management, but they are not the core task description consumed by `planner_llm`.

5.1 Contract 3: Grounded Context

The planner receives a compact, JSON-native view of relevant world state. This is best described as a compact scene graph derived from the KB, continually updated by the perception pipeline (human/person detection and object perception). Raw KB triples, detector metadata, and tracker-derived details remain available through scene/KB evidence paths to skills, but they are not duplicated into every LLM prompt.

18

```

1 {
2   "grounded_context": {
3     "entities": [
4       {
5         "id": "cup_jrjic",
6         "label": "cup",
7         "kind": "object",
8         "class": "Cup",
9         "visible": true,
10        "relations": [
11          {"predicate": "dbp:color", "object": "blue"},
12          {"predicate": "oro:isOn", "object": "table_1"}
13        ]
14      },
15      {
16        "id": "anonymous_person_ehfbf",
17        "label": null,
18        "kind": "person",
19        "class": "Human",
20        "visible": true,
21        "relations": []
22      }
23    ]
24  }
25 }
```

Table 6: Fields in Contract 3: grounded context.

Field	Meaning
<code>id</code>	Stable entity handle used by planner steps and skill arguments. This is the preferred reference when a skill needs to target a known person, object, or location.
<code>label</code>	Human-readable name or class hint. It may be empty for anonymous people or entities that should not be assigned a personal name.
<code>kind</code>	Entity category, such as <code>object</code> , <code>person</code> , or another supported runtime category. This prevents people from being treated as generic objects.
<code>class</code>	Compact semantic class used by the planner for reasoning, such as <code>Cup</code> , <code>Human</code> , or <code>Table</code> .
<code>visible</code>	Indicates whether the entity is currently grounded in the latest accepted scene or KB state. It should not be treated as a guarantee that the entity will still be available at execution time.
<code>relations</code>	Bounded list of relevant semantic relations, such as colour, containment, support, proximity, or ownership-style facts. These relations add task-relevant context without exposing the full KB.

The compact view should omit detector score, image coordinates, backend, source, and recency by default. Those values are execution evidence, not planning semantics, unless a future skill explicitly needs them in its own action arguments.

5.2 Grounded Context Pipeline

JSON object passed (along the system prompt and skills registry) to the chatbot and planner. Acts as LLM facing grounded context representing current entities in the KB.

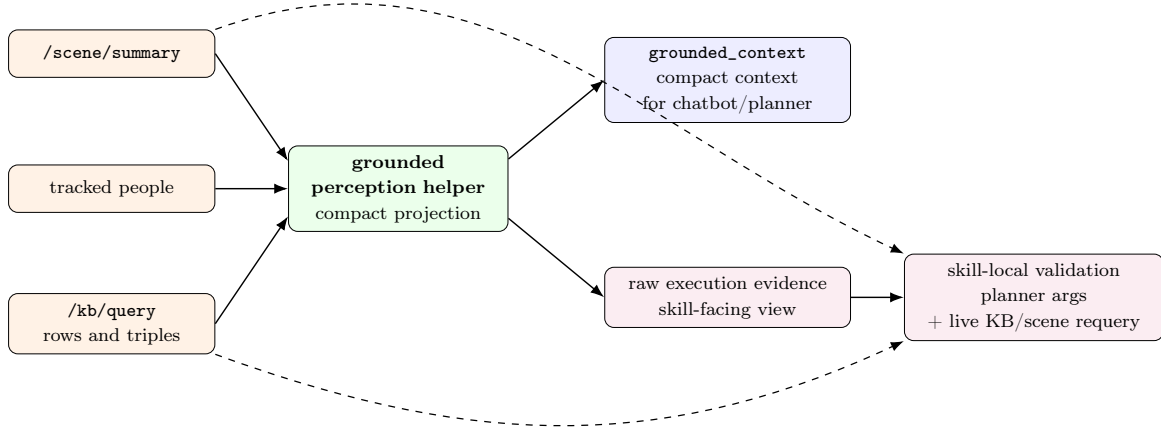


Figure 3: Grounded-context pipeline. Scene and KB evidence are projected by a grounded perception helper into a compact `grounded_context` for `chatbot_llm` and `planner_llm`. Skills receive planner-approved arguments through execution, while also retaining access to raw scene and KB evidence for live requery and validation before reporting outcomes.

Derivation rules:

1. Use text-derived entities only when unable to confirm entities with KB.
2. Keep anonymous people anonymous; do not invent personal names.
3. Keep IDs and labels separate.
4. Bound relation count per entity to keep prompt size manageable.

5.3 Planner LLM Internals

After the planner request is admitted, `planner_llm` builds a prompt from the compact request, compact grounding, the skill registry, allowed step types, and the output contract. Rule-based shortcuts may produce deterministic plans for simple motions. Otherwise, the LLM provider returns JSON, which is parsed and validated before publication. Invalid JSON or unsupported steps trigger a validation retry or a failure result.

The planner prompt should not contain raw detector coordinates, full RDF dumps, or stale perceptual reports. Those belong to skills and execution evidence.

5.4 Contract 4: Planner Output

The planner output is a plan: a sequence of skill steps to achieve the user goal. It should focus on executable content and minimal lineage. Communication policy and retry metadata are still valid runtime fields when used by the stack, but this controlled version foregrounds the ordered `steps` list because that is the essential planner output.

18

```
1 {
2   "plan": {
3     "goal_id": "goal_turn_123",
4     "plan_id": "plan_123",
5     "plan_version": 1,
6     "status": "planned",
7     "steps": [
8       {
9         "id": "step_1",
10        "type": "skill",
11        "name": "look_at",
12        "args": {"target_frame": "anonymous_person_ehfbf"},
13        "requires": [],
14        "on_failure": "replan"
15      }
16    ]
17  }
18 }
```

Table 7: Fields in Contract 4: planner output.

Field	Meaning
<code>goal_id</code>	User objective that this plan attempts to satisfy. It links the planner output to the active goal.
<code>plan_id</code>	Identifier for this concrete plan proposal. It is useful when the runtime keeps plan history or compares revised plans for the same goal.
<code>plan_version</code>	Revision number for replans of the same goal. This helps distinguish the initial plan from later planner-supervision outputs.
<code>status</code>	Planning outcome for this plan, such as <code>planned</code> , <code>needs_clarification</code> , or <code>failed_to_plan</code> . This describes whether <code>planner_llm</code> produced an executable plan, not whether the robot has already executed it.
<code>steps</code>	Ordered list of skill calls produced by the planner. This is the essential planner output consumed by <code>nao_orchestrator</code> .
<code>steps[].id</code>	Stable step identifier used for execution feedback, dependency tracking, and possible replan joins.
<code>steps[].name</code>	Skill name selected from the allowed skill registry. The planner should not invent skills or use robot-specific APIs directly.
<code>steps[].args</code>	Skill arguments. They should refer to known entity IDs, explicit primitive values, or arguments supported by the selected skill schema.
<code>steps[].requires</code>	Dependencies between steps, if any. This allows the runtime to know which steps depend on previous successful outcomes.
<code>steps[].on_failure</code>	Recovery policy proposed by <code>planner_llm</code> for this step. If the step fails, <code>nao_orchestrator</code> may fail safely, continue if allowed, request user input, or send feedback back to <code>planner_llm</code> for replanning. Allowed values include <code>replan</code> , <code>clarify</code> , <code>ask_user</code> , <code>continue</code> , <code>ignore</code> , and <code>fail</code> .

Runtime recovery metadata. The current contract supports recovery metadata as skill failure may require controlled recovery. Each step may carry a `retry_budget`, and the plan envelope may also carry a plan-level `retry_budget`. This does not mean that `retry` is a separate `on_failure` value. Instead, retry behaviour is controlled by `nao_orchestrator` using the available retry budget together with the step’s `on_failure` policy. The plan envelope may also carry `communication_policy`. This controls when the planner/dialogue path should communicate progress, completion, clarification needs, or failure.

When a step fails with `on_failure = replan`, `nao_orchestrator` halts or blocks continuation, publishes execution feedback to `planner_llm`, and waits for a revised validated plan before dispatching further actions. This keeps execution authority inside `nao_orchestrator`, while allowing recovery to be driven by planner supervision rather than by hard-coded local replanning.

These fields are included because they support runtime recovery, traceability, and human-robot interaction, but the essential planner output remains the ordered **steps** list. If the planner cannot produce a plan, the output should represent that as a planning result rather than filling the plan with ambiguous failure fields. For the simple version, **steps** is the most important field.

5.5 Contract 5: Execution Feedback

For the simple version, execution feedback can be minimal. The planner needs to know which goal is affected, which step failed or succeeded, and a compact result if that result is useful for replanning or dialogue. A larger payload is justified only when the planner supervision logic, orchestrator, traces, or debug tooling use the extra fields.

18

```

1 {
2   "goal_id": "goal_turn_123",
3   "status": "failed",
4   "step": {
5     "id": "step_1",
6     "name": "look_at"
7   },
8   "reason": "target frame unavailable",
9   "result_payload": {
10    "target_found": false,
11    "target_kind": "person"
12  }
13 }
```

Runtime feedback may include additional fields used by `nao_orchestrator`, logging, and planner supervision. Examples include `plan_id`, `plan_version`, `event_type`, `validation_status`, `blocking`, `needs_user_input`, `timestamp_sec`, `result_summary`, and retry metadata. These fields are useful for execution control and debugging, but the minimal planner-facing feedback is the goal, step, status, reason, and result payload.

5.6 Contract 6: Planner Dialogue Act

Planner dialogue acts represent planner intent to communicate, not direct speech execution. They are relayed through the orchestrator and realized by the dialogue layer.

```

1 {
2   "goal_id": "goal_turn_123",
3   "act": "ask_clarification",
4   "await_user_response": true,
```

```

5  "reason": "multiple visible people",
6  "text_hint": "Which person should I look at?",
7  "slots_needed": ["target_person"]
8  }

```

Allowed dialogue acts are acknowledgement, progress update, clarification, help request, failure explanation, completion notification, and cancellation notification. The planner does not speak directly; it emits a structured communication request.

6 Implementation

6.1 Repository Structure

Package	Role in thesis implementation
dialogue_manager	Dialogue lifecycle and speaking ownership.
chatbot_llm	LLM dialogue backend, response generation, task intent declaration, KB snapshot injection.
planner_common	Contract dataclasses, JSON normalization, shared constants.
planner_llm	Goal supervision, plan generation, replanning, dialogue acts.
nao_orchestrator	Deterministic validation, ordered execution, execution feedback.
kb_skills	Query/revise boundary to KnowledgeCore.
nao_scene_grounding	Detector-to-KB object grounding and scene summaries.
nao_replay_motion, nao_look_at, nao_say_skill	NAO-specific skill adapters.

Table 8: Main repository packages used in the thesis.

6.2 Planner Engine

TODO: Describe `planner_llm`: provider, prompt construction, skill registry, JSON extraction, validation, fallback behaviour.

6.3 Goal-Keyed Supervisor

TODO: Describe active goal state, plan versions, feedback handling, retry budgets, waiting-user state, and cancellation/supersession.

6.4 Skill Registry

TODO: Describe abstract skill metadata. Include current skills: `perform_motion`, `look_at`, `scan`. Explain why `scan` is a composite/perception skill.

Skill	Category	Expected use
<code>perform_motionembodiment</code>		Posture and head motion through NAO adapters.
<code>look_at</code>	attention	Direct gaze toward a target frame or reset gaze.
<code>scan</code>	perception/composite	Refresh scene information and expose observations to the planner.

Table 9: Initial planner-facing skill repertoire.

6.5 Orchestrator and Execution Boundary

TODO: Explain `nao\_orchestrator` plan validation, ordered execution, step events, and feedback. Emphasise that the LLM never executes robot skills directly.

6.6 Grounding and Knowledge

TODO: Describe `nao_scene_grounding`, `/scene/summary`, `kb_skills`, and `knowledge_snapshot`.

6.7 Models and Runtime Configuration

TODO: Document model candidates and final model choice. Include practical constraints such as JSON validity, latency, cloud quota, and local fallback.

7 Validation Methodology

7.1 Validation Goals

The validation should test whether the architecture works as a planner-mediated HRI system, not merely whether a demo command succeeds once.

Metric	Meaning
Plan validity	Whether the LLM output satisfies the schema and skill registry.
Task completion	Whether the robot executes all required steps successfully.
Multi-intent coverage	Whether all requested subgoals are represented in the plan.
Recovery behaviour	Whether failures trigger replan, clarification, or safe failure.
Interaction quality	Whether user-facing responses are timely and appropriate.
Latency	Time from user request to plan, execution start, and completion.
Modularity evidence	Whether components can be replaced or disabled without collapsing the stack.

Table 10: Proposed validation metrics.

7.2 Scenario Matrix

ID	Scenario	Expected behaviour
S1	Atomic action: “look left”	Direct or planner-mediated valid <code>perform_motion</code> step.
S2	Two-step action: “stand up and look left”	Ordered plan with two executable steps.
S3	Perception query: “what can you see?”	Planner uses <code>scan</code> , consumes scene/KB context, reports via dialogue act.
S4	Scene-dependent task: “look at the cup”	Planner grounds target and emits valid <code>look_at</code> step or asks clarification.
S5	Ambiguous instruction: “look at that”	Planner requests clarification.
S6	Failure case: unavailable action server	Orchestrator emits failure, planner replans or explains failure.
S7	Multi-intent over time	User updates goal during execution; planner supersedes or updates active goal.
S8	Dialogue-only turn: “hello, who are you?”	Chatbot responds without generating executable intent.

Table 11: Initial validation scenario matrix.

7.3 Ablation Plan

- Direct intent execution vs planner-mediated execution.
- Planner with minimal prompt vs planner with generated skill harness.
- Without execution feedback vs with execution feedback.
- Without grounded context vs with knowledge/scene context.
- Single-intent vs multi-intent requests.

7.4 Data Collection

TODO: Define log capture procedure: ROS bags or topic echoes, timestamps, planner JSON, feedback events, dialogue acts, task success labels.

Listing 1: Runtime topics to capture during validation.

```

1 ros2 topic echo /planner/request
2 ros2 topic echo /intents
3 ros2 topic echo /planner/execution_feedback
4 ros2 topic echo /planner/dialogue_act
5 ros2 topic echo /scene/summary

```

8 Results

TODO: This section will remain empty until validation is complete. Use the template below.

8.1 Plan Validity Results

Scenario	Trials	Valid plans	Completed	Mean latency
S1 Atomic action	—	—	—	—
S2 Two-step action	—	—	—	—
S3 Perception query	—	—	—	—
S4 Scene-dependent task	—	—	—	—

Table 12: Results template for validation trials.

8.2 Qualitative Scenario Traces

TODO: Paste short traces of one successful and one failed/recovered interaction.

9 Discussion

9.1 Architectural Findings

TODO: Discuss whether modular separation helped. Use examples from validation.

9.2 Planner Behaviour

TODO: Discuss JSON validity, skill selection, multi-intent decomposition, and common failure modes.

9.3 Grounding and World State

TODO: Discuss whether KB/scene grounding helped. Explain remaining gap toward WME.

9.4 ROS4HRI Modularity

TODO: Discuss how much of the architecture is robot-agnostic vs NAO-specific.

10 Limitations and Future Work

10.1 Current Limitations

- The physical implementation is centred on NAO, so cross-robot portability is architectural rather than fully proven.
- LLM planning remains sensitive to prompt design, model availability, and JSON schema adherence.
- Validation is scenario-based and cannot prove exhaustive safety.
- Perception and grounding quality limit the planner’s world-state awareness.
- Real-time performance depends on model latency, cloud availability, and local compute constraints.

10.2 Future Work

- migrate planner-gate ownership fully into `nao_orchestrator`;
- generate planner prompt packs from the skill registry;
- expand the skill repertoire and add arm/gesture behaviours safely;
- integrate richer VLM-based scene understanding;
- develop the World Model Enricher as a persistent, uncertainty-aware context layer;
- test the same planner interface on another ROS-compatible robot or simulator.

11 Conclusion

TODO: Write after results. Restate contribution, answer research questions, and explain what was learned about modular LLM planning in ROS4HRI.

References

- [1] D. McDermott et al., “PDDL: The planning domain definition language,” Yale Center for Computational Vision and Control, Tech. Rep. CVC TR-98-003/DCS TR-1165, 1998.
- [2] K. Erol, J. Hendler, and D. S. Nau, “HTN planning: Complexity and expressivity,” in *Proceedings of the Twelfth National Conference on Artificial Intelligence*, AAAI Press, 1994, pp. 1123–1128.
- [3] D. S. Nau et al., “SHOP2: An HTN planning system,” *Journal of Artificial Intelligence Research*, vol. 20, pp. 379–404, 2003.
- [4] I. Georgievski and M. Aiello, “An overview of hierarchical task network planning,” *arXiv preprint arXiv:1403.7426*, 2014.
- [5] B. Liu et al., “LLM+P: Empowering large language models with optimal planning proficiency,” *arXiv preprint arXiv:2304.11477*, 2023.
- [6] K. Valmeekam, M. Marquez, S. Sreedharan, and S. Kambhampati, “On the planning abilities of large language models: A critical investigation,” *arXiv preprint arXiv:2305.15771*, 2023.
- [7] M. Ahn et al., “Do as i can, not as i say: Grounding language in robotic affordances,” *arXiv preprint arXiv:2204.01691*, 2022.
- [8] T. Fong, I. Nourbakhsh, and K. Dautenhahn, “A survey of socially interactive robots,” *Robotics and Autonomous Systems*, vol. 42, no. 3–4, pp. 143–166, 2003.
- [9] M. A. Goodrich and A. C. Schultz, “Human-robot interaction: A survey,” *Foundations and Trends in Human-Computer Interaction*, vol. 1, no. 3, pp. 203–275, 2007.
- [10] M. Quigley et al., “ROS: An open-source robot operating system,” in *ICRA Workshop on Open Source Software*, 2009.
- [11] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot operating system 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, eabm6074, 2022.
- [12] Y. Mohamed and S. Lemaignan, “ROS for human-robot interaction,” *arXiv preprint arXiv:2012.13944*, 2020.
- [13] D. Gouaillier et al., “The NAO humanoid: A combination of performance and affordability,” *arXiv preprint arXiv:0807.3223*, 2008.
- [14] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, “Language models as zero-shot planners: Extracting actionable knowledge for embodied agents,” *arXiv preprint arXiv:2201.07207*, 2022.
- [15] J. Liang et al., “Code as policies: Language model programs for embodied control,” *arXiv preprint arXiv:2209.07753*, 2022.

- [16] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2022.
- [17] W. Huang et al., “Inner monologue: Embodied reasoning through planning with language models,” *arXiv preprint arXiv:2207.05608*, 2022.
- [18] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” *arXiv preprint arXiv:2303.11366*, 2023.
- [19] J. Li, D. Li, S. Savarese, and S. Hoi, “Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models,” *arXiv preprint arXiv:2301.12597*, 2023.
- [20] H. Liu, C. Li, Q. Wu, and Y. J. Lee, “Visual instruction tuning,” *arXiv preprint arXiv:2304.08485*, 2023.
- [21] D. Driess et al., “Palm-e: An embodied multimodal language model,” *arXiv preprint arXiv:2303.03378*, 2023.
- [22] A. Brohan et al., “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” *arXiv preprint arXiv:2307.15818*, 2023.
- [23] M. J. Kim et al., “Openvla: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [24] M. Tenorth and M. Beetz, “KnowRob: A knowledge processing infrastructure for cognition-enabled robots,” *The International Journal of Robotics Research*, vol. 32, no. 5, pp. 566–590, 2013.
- [25] M. Tenorth and M. Beetz, “Representations for robot knowledge in the KnowRob framework,” *Artificial Intelligence*, vol. 247, pp. 151–169, 2017.
- [26] A. Saxena, A. Jain, O. Sener, A. Jami, D. K. Misra, and H. S. Koppula, “RoboBrain: Large-scale knowledge engine for robots,” *arXiv preprint arXiv:1412.0691*, 2014.
- [27] E. Bastianelli, D. D. Bloisi, R. Capobianco, G. Gemignani, L. Iocchi, and D. Nardi, “Knowledge representation for robots through human-robot interaction,” *arXiv preprint arXiv:1307.7351*, 2013.
- [28] L. Grassi, C. T. Recchiuto, and A. Sgorbissa, “Knowledge triggering, extraction and storage via human-robot verbal interaction,” *arXiv preprint arXiv:2104.11170*, 2021.
- [29] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, “Language models as zero-shot planners: Extracting actionable knowledge for embodied agents,” in *International Conference on Machine Learning*, 2022.
- [30] Open Robotics, *Robot operating system 2 documentation*, Accessed as project background documentation, 2025.
- [31] ROS4HRI Community, *Ros4hri: Ros for human-robot interaction*, Accessed as project background documentation, 2025.

A Appendix A: Full Runtime Contracts

TODO: Move final JSON schemas here once stable.

B Appendix B: Launch Commands

```

1 ros2 launch nao_chatbot nao_chatbot_sim_demo.launch.py \
2   start_planner_llm:=true \
3   chatbot_planner_mode_enabled:=true \
4   planner_llm_think:=false

```

C Appendix C: Validation Log Template

Field	Value
Scenario ID	TODO: S1–S8
User command	TODO: Paste command
Model	TODO: Model name
Planner request	TODO: Path or pasted payload
Planner output	TODO: Path or pasted payload
Execution feed-back	TODO: Path or pasted payload
Dialogue act	TODO: Path or pasted payload
Outcome	TODO: success / failure / recovered / clarification
Notes	TODO: Observations

Table 13: Validation log template.